8.5.2 Deleting data

The scaffolded pages include one that enables you to delete an entity. The `OnPostAysnc` method in the `DeleteModel` class contains the code that actually removes the entry from the database. It is important to understand how it works because it is suboptimal.

Listing 8.33 The scaffolded `OnPostAsync` method in the `DeleteModel`

```
public async Task<IActionResult> OnPostAsync(int? id)
{
    if (id == null)                                          ❶
    {                                                        ❶
        return NotFound();                                   ❶
    }                                                        ❶

    Property = await _context.Properties.FindAsync(id);      ❷

    if (Property != null)
    {
        _context.Properties.Remove(Property);                ❸
        await _context.SaveChangesAsync();                   ❹
    }

    return RedirectToPage("./Index");
}
```

❶ Checks that a key value was passed to the method

❷ This retrieves the matching entry from the database. The context starts tracking it.

❸ The DbSet.Remove method sets the entity's state to Deleted.

❹ SaveChangesAsync commits the changes to the database.

We have already discussed how to use route constraints to replace the null check at the beginning of this method. The other suboptimal feature of the scaffolded code is that it results in *two* commands being executed against the database. The first command retrieves the item from the database, so the context can start tracking it. The code uses the `DbSet.Remove` method to set the entity's state as `Deleted`. The second command, executed when